

DEVOPS MASTER COURSE

Volume 12B

AWS Enterprise Cloud Engineering

*Organizations • IAM • Networking • Compute • Data • Security • Reliability •
Operations • FinOps*

Chapters 11-20 | 2026 Edition

Beginner to Enterprise Cloud Architecture and Operations

About This Part

Part B develops provider-depth in Amazon Web Services. It begins with the multi-account foundation and proceeds through identity, network, compute, data, security, reliability, operations and cost before integrating them into an enterprise landing-zone capstone.

2026 platform note: The 2026 edition uses AWS Organizations and AWS Control Tower as the landing-zone foundation, IAM Identity Center for workforce federation, short-lived workload identity, organization-wide evidence and explicit regional recovery testing.

Volume 12 Roadmap

Part	Coverage	Status
12A	Multi-cloud architecture and landing-zone foundations	Completed
12B	AWS enterprise cloud engineering	Completed in this package
12C	Azure enterprise cloud engineering	Course sequence
12D	Google Cloud enterprise engineering	Course sequence
12E	Hybrid and multi-cloud operations, migration, DR, security and FinOps	Course sequence
12F	Enterprise projects, certifications, interviews and final capstone	Course sequence

Scope, Safety and Cost Controls

- Use disposable sandboxes, free tiers, local simulations or approved training subscriptions. Configure budgets and expiry before creating resources.
- Never use production identities, customer data, employer account structures, internal IP ranges or confidential contracts in course artifacts.
- Cloud services, prices, limits, recommendations and certifications change. Verify official documentation before production use or exam registration.
- Run failure, security and recovery tests only in approved nonproduction environments with explicit stop conditions.
- Destroy resources, revoke credentials, remove local state copies and verify billing inventory after every lab.

Learning Outcomes

- Explain and design the major architecture domains covered by AWS Enterprise Cloud Engineering.
- Implement provider or cross-cloud controls through version-controlled automation.
- Test identity, network, security, reliability, operations and cost behavior.
- Troubleshoot failures using effective policy, route, service and telemetry evidence.
- Produce sanitized architecture, runbook, laboratory and interview portfolio artifacts.

Table of Contents

Chapter 11 AWS Organizations, Control Tower and Multi-Account Foundations

Chapter 12 AWS Identity, Federation and Organization Guardrails

Chapter 13 AWS Networking, DNS and Hybrid Connectivity

Chapter 14 AWS Compute, Containers and Serverless Architecture

Chapter 15 AWS Storage, Databases and Data Protection

Chapter 16 AWS Security Engineering and Detection

Chapter 17 AWS Reliability, Multi-Region Design and Disaster Recovery

Chapter 18 AWS Observability, Systems Management and Incident Operations

Chapter 19 AWS FinOps, Governance and Cost Engineering

Chapter 20 AWS Enterprise Landing-Zone Capstone

AWS Enterprise Architecture and Operations Reference

Completion Checklist

Additional Portfolio Projects

30-Day Practice Plan

Glossary

Official Documentation Index

Chapter 11: AWS Organizations, Control Tower and Multi-Account Foundations

Building a governed account estate before workloads scale

Why this chapter matters: AWS accounts are strong isolation, billing and policy boundaries. A clear organization and landing-zone lifecycle prevents root-account exposure, inconsistent controls and unowned spend.

Learning Objectives

- Explain the architecture, responsibility boundaries and major trade-offs of aws organizations and organizational units.
- Design aws control tower landing-zone lifecycle for enterprise scale, least privilege and bounded failure.
- Implement account factory and vending automation through version-controlled automation and policy checks.
- Troubleshoot delegated administration and evidence accounts using provider-native telemetry and repeatable evidence.
- Deliver a sanitized portfolio artifact for design and validate an aws multi-account landing zone.

Core Concepts

AWS Organizations and organizational units

AWS Organizations and organizational units is a design boundary, not merely a console feature. In AWS Organizations, Control Tower and Multi-Account Foundations, the team must document ownership, trust assumptions, dependency direction, failure impact and the evidence used to prove the control. The recommended implementation uses AWS Organizations / OUs, AWS Control Tower, Account Factory, Log Archive and Audit accounts only where they satisfy an explicit requirement; equivalent services are not automatically interchangeable.

AWS Control Tower landing-zone lifecycle

A production implementation of AWS Control Tower landing-zone lifecycle separates the management plane from workload execution and constrains blast radius. Architecture reviews should evaluate quotas, regional behavior, identity propagation, private connectivity, encryption, logging and recovery. The operational contract must state who changes the control, who monitors it and how an emergency exception expires.

Account Factory and vending automation

For Account Factory and vending automation, repeatability matters more than a one-time successful deployment. Infrastructure as code, policy tests, deployment verification, drift detection and rollback evidence should be retained together. A design is incomplete when it works only for the original engineer or depends on undocumented console steps.

Delegated administration and evidence accounts

The strongest validation for Delegated administration and evidence accounts combines an allowed test, a denied test and a failure test. Teams should prove normal workload behavior, demonstrate that prohibited behavior is blocked and exercise recovery under a bounded fault. Metrics, logs, audit events and cost records become part of the acceptance package.

Service and Capability Map

Capability	Service or pattern	Enterprise use
Hierarchy	AWS Organizations / OUs	Apply SCPs, delegated administration and workload grouping.
Landing zone	AWS Control Tower	Establish governed multi-account baselines and registered OUs.

Capability	Service or pattern	Enterprise use
Account vending	Account Factory	Create standardized workload accounts with metadata and lifecycle.
Audit evidence	Log Archive and Audit accounts	Protect organization-wide logs and security administration.
Inventory	AWS Config aggregator	Evaluate configuration across accounts and Regions.
Identity	IAM Identity Center	Federate workforce access into accounts and permission sets.

Architecture and Operating Decisions

- Use AWS managed services only after recording availability, security, portability and cost consequences.
- Separate production, nonproduction, security and shared services with strong administrative and billing boundaries.
- Prefer short-lived workload identity, private service access and centralized audit evidence over persistent credentials and public administration.
- Automate account factory and vending automation with reviewed plans, policy tests, staged rollout and post-deployment verification.
- Define recovery objectives, quotas, ownership and decommissioning before the service becomes business critical.

Reference Configuration or Decision Pattern

```
organization:
  roots: [enterprise]
  ous: [security, infrastructure, workloads-prod, workloads-nonprod, sandbox]
  shared_accounts: [management, log-archive, audit, network]
  account_vending:
    required: [owner, cost_center, environment, data_classification, expiry]
    baseline: [cloudtrail, config, guardrails, budgets, identity-center]
    closure: [quarantine, export_evidence, remove_access, close_account]
```

Interpretation: Treat the pattern as a starting point. Replace placeholders with measured requirements, provider limits and approved organizational standards.

Production Readiness Checklist

- ☐ Architecture diagrams and responsibility boundaries for aws organizations, control tower and multi-account foundations are current and version controlled.
- ☐ Identity uses groups, roles, workload federation and privileged elevation with no unnecessary standing administrator access.
- ☐ Network paths, DNS behavior, encryption, audit logging and service limits are tested before production use.
- ☐ Changes are delivered through IaC with validation, policy, approval, rollback and drift detection.
- ☐ Recovery, cost allocation, ownership metadata and teardown have measurable evidence.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Deployment succeeds but service is unreachable	Route, firewall, DNS or private-endpoint path is incomplete	Trace the path hop by hop; compare route, policy, DNS and endpoint state; then run an explicit reachability test.	Successful synthetic request and flow/audit evidence
Authorization is broader than intended	Inherited role, wildcard permission or unmanaged credential	Inspect effective permissions, credential age and access logs; replace broad grants with scoped roles and short-lived identity.	Effective-access review and denied-action test
Regional failure breaks the workload	State, dependency or failover mechanism remained single-region	Map stateful dependencies, restore order and traffic controls; exercise failover and fallback against measured RTO/RPO.	Timed recovery log with data-integrity checks

Symptom	Likely cause	Corrective action	Validation evidence
Cost rises without traffic growth	Idle capacity, data transfer, logging volume or commitment mismatch	Break down spend by owner, service, region and usage driver; validate rightsizing or commitment changes against SLOs.	Cost delta plus performance and reliability validation

Hands-on Lab 11: Design and Validate an AWS Multi-Account Landing Zone

Prerequisites

- Disposable provider sandbox or a local simulation.
- Terraform or provider CLI, Git and a diagramming tool.
- Synthetic identities and data; no production credentials or confidential information.

Implementation

1. Create the architecture and responsibility model for aws organizations, control tower and multi-account foundations using fictional enterprise requirements.
2. Implement the smallest safe sandbox using AWS Organizations / OUs, AWS Control Tower, Account Factory; use free or low-cost options where available.
3. Apply identity, network, encryption, logging, metadata and budget controls through IaC or an equivalent version-controlled simulation.
4. Run one approved behavior test and one denied behavior test; save the commands, timestamps and resulting telemetry.
5. Inject a bounded configuration or dependency failure, follow the runbook, recover and record measured recovery time.
6. Destroy or quarantine the lab, revoke temporary access and verify that no unexpected billable resources remain.

Validation Evidence

- Version-controlled architecture, IaC and decision record.
- Allowed, denied and failure-test outputs with timestamps.
- Cost, recovery, ownership and teardown verification.

Cleanup

- Destroy test resources in dependency order and verify the provider inventory and billing view.
- Revoke temporary identities, remove local state copies and sanitize all portfolio evidence.

Interview Questions and Model Answers

1. What is the most important design principle in AWS Organizations, Control Tower and Multi-Account Foundations?

Make responsibility and failure boundaries explicit, then automate controls and prove them with normal, denied and recovery tests rather than trusting a successful deployment alone.

2. How would you choose between the main AWS services in this domain?

Start with workload requirements for latency, state, scale, compliance, team capability, recovery and cost. Select the managed service whose operating model fits those requirements and document the consequences in an ADR.

3. What evidence would you require before production approval?

Architecture and threat models, reviewed IaC plan, effective-access results, network and encryption tests, known audit events, SLO and recovery validation, ownership metadata, budget routing and rollback evidence.

4. How do you reduce provider lock-in without creating a lowest-common-denominator platform?

Preserve portable source, data, interfaces and deployment artifacts where a credible exit scenario requires them, while still using provider-specific capabilities when their measurable value exceeds migration and dependency risk.

Chapter Summary

- AWS Organizations and organizational units defines a key architecture boundary.
- AWS Control Tower landing-zone lifecycle must be automated and least-privileged.
- Account Factory and vending automation requires policy, telemetry and recovery evidence.
- Delegated administration and evidence accounts should be tested before production and after material change.

Chapter 12: AWS Identity, Federation and Organization Guardrails

Controlling humans, workloads and accounts without permanent broad credentials

Why this chapter matters: Identity is the highest-leverage AWS control. Federation, permission sets, SCPs, resource policies and workload roles must work together without treating any one mechanism as a complete authorization model.

Learning Objectives

- Explain the architecture, responsibility boundaries and major trade-offs of workforce federation with iam identity center.
- Design iam roles, permission boundaries and session controls for enterprise scale, least privilege and bounded failure.
- Implement service control policies and resource control through version-controlled automation and policy checks.
- Troubleshoot workload identity for ec2, ecs, eks and ci/cd using provider-native telemetry and repeatable evidence.
- Deliver a sanitized portfolio artifact for implement federated aws access and guardrail tests.

Core Concepts

Workforce federation with IAM Identity Center

Workforce federation with IAM Identity Center is a design boundary, not merely a console feature. In AWS Identity, Federation and Organization Guardrails, the team must document ownership, trust assumptions, dependency direction, failure impact and the evidence used to prove the control. The recommended implementation uses IAM Identity Center, IAM roles / IRSA / EKS Pod Identity, Service control policies, Permission boundaries only where they satisfy an explicit requirement; equivalent services are not automatically interchangeable.

IAM roles, permission boundaries and session controls

A production implementation of IAM roles, permission boundaries and session controls separates the management plane from workload execution and constrains blast radius. Architecture reviews should evaluate quotas, regional behavior, identity propagation, private connectivity, encryption, logging and recovery. The operational contract must state who changes the control, who monitors it and how an emergency exception expires.

Service control policies and resource control

For Service control policies and resource control, repeatability matters more than a one-time successful deployment. Infrastructure as code, policy tests, deployment verification, drift detection and rollback evidence should be retained together. A design is incomplete when it works only for the original engineer or depends on undocumented console steps.

Workload identity for EC2, ECS, EKS and CI/CD

The strongest validation for Workload identity for EC2, ECS, EKS and CI/CD combines an allowed test, a denied test and a failure test. Teams should prove normal workload behavior, demonstrate that prohibited behavior is blocked and exercise recovery under a bounded fault. Metrics, logs, audit events and cost records become part of the acceptance package.

Service and Capability Map

Capability	Service or pattern	Enterprise use
Workforce access	IAM Identity Center	Map enterprise groups to account permission sets.
Workload identity	IAM roles / IRSA / EKS Pod Identity	Issue short-lived credentials to software.

Capability	Service or pattern	Enterprise use
Organization guardrail	Service control policies	Limit maximum permissions across accounts.
Delegation guardrail	Permission boundaries	Constrain roles created by delegated teams.
Resource trust	Resource-based policies	Authorize cross-account use of selected resources.
Secrets	Secrets Manager / Parameter Store	Keep secret material out of code and pipelines.

Architecture and Operating Decisions

- Use AWS managed services only after recording availability, security, portability and cost consequences.
- Separate production, nonproduction, security and shared services with strong administrative and billing boundaries.
- Prefer short-lived workload identity, private service access and centralized audit evidence over persistent credentials and public administration.
- Automate service control policies and resource control with reviewed plans, policy tests, staged rollout and post-deployment verification.
- Define recovery objectives, quotas, ownership and decommissioning before the service becomes business critical.

Reference Configuration or Decision Pattern

```
authorization_layers:
  identity_policy: grants_requested_actions
  permissions_boundary: caps_delegated_roles
  scp: caps_account_permissions
  resource_policy: grants_resource_trust
  session_policy: reduces_temporary_session
rules:
  - no_long_lived_ci_keys
  - mfa_for_privileged_humans
  - break_glass_isolated_and_tested
  - deny_unapproved_regions_with_exceptions
```

Interpretation: Treat the pattern as a starting point. Replace placeholders with measured requirements, provider limits and approved organizational standards.

Production Readiness Checklist

- ☐ Architecture diagrams and responsibility boundaries for aws identity, federation and organization guardrails are current and version controlled.
- ☐ Identity uses groups, roles, workload federation and privileged elevation with no unnecessary standing administrator access.
- ☐ Network paths, DNS behavior, encryption, audit logging and service limits are tested before production use.
- ☐ Changes are delivered through IaC with validation, policy, approval, rollback and drift detection.
- ☐ Recovery, cost allocation, ownership metadata and teardown have measurable evidence.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Deployment succeeds but service is unreachable	Route, firewall, DNS or private-endpoint path is incomplete	Trace the path hop by hop; compare route, policy, DNS and endpoint state; then run an explicit reachability test.	Successful synthetic request and flow/audit evidence
Authorization is broader than intended	Inherited role, wildcard permission or unmanaged credential	Inspect effective permissions, credential age and access logs; replace broad grants with scoped roles and short-lived identity.	Effective-access review and denied-action test
Regional failure breaks the workload	State, dependency or failover mechanism remained single-region	Map stateful dependencies, restore order and traffic controls; exercise failover and fallback against measured RTO/RPO.	Timed recovery log with data-integrity checks

Symptom	Likely cause	Corrective action	Validation evidence
Cost rises without traffic growth	Idle capacity, data transfer, logging volume or commitment mismatch	Break down spend by owner, service, region and usage driver; validate rightsizing or commitment changes against SLOs.	Cost delta plus performance and reliability validation

Hands-on Lab 12: Implement Federated AWS Access and Guardrail Tests

Prerequisites

- Disposable provider sandbox or a local simulation.
- Terraform or provider CLI, Git and a diagramming tool.
- Synthetic identities and data; no production credentials or confidential information.

Implementation

1. Create the architecture and responsibility model for aws identity, federation and organization guardrails using fictional enterprise requirements.
2. Implement the smallest safe sandbox using IAM Identity Center, IAM roles / IRSA / EKS Pod Identity, Service control policies; use free or low-cost options where available.
3. Apply identity, network, encryption, logging, metadata and budget controls through IaC or an equivalent version-controlled simulation.
4. Run one approved behavior test and one denied behavior test; save the commands, timestamps and resulting telemetry.
5. Inject a bounded configuration or dependency failure, follow the runbook, recover and record measured recovery time.
6. Destroy or quarantine the lab, revoke temporary access and verify that no unexpected billable resources remain.

Validation Evidence

- Version-controlled architecture, IaC and decision record.
- Allowed, denied and failure-test outputs with timestamps.
- Cost, recovery, ownership and teardown verification.

Cleanup

- Destroy test resources in dependency order and verify the provider inventory and billing view.
- Revoke temporary identities, remove local state copies and sanitize all portfolio evidence.

Interview Questions and Model Answers

1. What is the most important design principle in AWS Identity, Federation and Organization Guardrails?

Make responsibility and failure boundaries explicit, then automate controls and prove them with normal, denied and recovery tests rather than trusting a successful deployment alone.

2. How would you choose between the main AWS services in this domain?

Start with workload requirements for latency, state, scale, compliance, team capability, recovery and cost. Select the managed service whose operating model fits those requirements and document the consequences in an ADR.

3. What evidence would you require before production approval?

Architecture and threat models, reviewed IaC plan, effective-access results, network and encryption tests, known audit events, SLO and recovery validation, ownership metadata, budget routing and rollback evidence.

4. How do you reduce provider lock-in without creating a lowest-common-denominator platform?

Preserve portable source, data, interfaces and deployment artifacts where a credible exit scenario requires them, while still using provider-specific capabilities when their measurable value exceeds migration and dependency risk.

Chapter Summary

- Workforce federation with IAM Identity Center defines a key architecture boundary.
- IAM roles, permission boundaries and session controls must be automated and least-privileged.
- Service control policies and resource control requires policy, telemetry and recovery evidence.
- Workload identity for EC2, ECS, EKS and CI/CD should be tested before production and after material change.

Chapter 13: AWS Networking, DNS and Hybrid Connectivity

Designing VPCs, transit, private services and predictable routing

Why this chapter matters: AWS network designs fail when CIDR, routes, DNS, inspection and ownership are decided independently. Enterprise connectivity requires an IP plan, explicit traffic flows and tested return paths.

Learning Objectives

- Explain the architecture, responsibility boundaries and major trade-offs of vpc segmentation and ip address management.
- Design transit gateway and centralized inspection for enterprise scale, least privilege and bounded failure.
- Implement direct connect, vpn and hybrid routing through version-controlled automation and policy checks.
- Troubleshoot route 53, privatelink and controlled ingress/egress using provider-native telemetry and repeatable evidence.
- Deliver a sanitized portfolio artifact for build an aws hub-and-spoke network with private service access.

Core Concepts

VPC segmentation and IP address management

VPC segmentation and IP address management is a design boundary, not merely a console feature. In AWS Networking, DNS and Hybrid Connectivity, the team must document ownership, trust assumptions, dependency direction, failure impact and the evidence used to prove the control. The recommended implementation uses Amazon VPC, VPC IPAM, AWS Transit Gateway, AWS PrivateLink only where they satisfy an explicit requirement; equivalent services are not automatically interchangeable.

Transit Gateway and centralized inspection

A production implementation of Transit Gateway and centralized inspection separates the management plane from workload execution and constrains blast radius. Architecture reviews should evaluate quotas, regional behavior, identity propagation, private connectivity, encryption, logging and recovery. The operational contract must state who changes the control, who monitors it and how an emergency exception expires.

Direct Connect, VPN and hybrid routing

For Direct Connect, VPN and hybrid routing, repeatability matters more than a one-time successful deployment. Infrastructure as code, policy tests, deployment verification, drift detection and rollback evidence should be retained together. A design is incomplete when it works only for the original engineer or depends on undocumented console steps.

Route 53, PrivateLink and controlled ingress/egress

The strongest validation for Route 53, PrivateLink and controlled ingress/egress combines an allowed test, a denied test and a failure test. Teams should prove normal workload behavior, demonstrate that prohibited behavior is blocked and exercise recovery under a bounded fault. Metrics, logs, audit events and cost records become part of the acceptance package.

Service and Capability Map

Capability	Service or pattern	Enterprise use
Network boundary	Amazon VPC	Isolate workload subnets, route tables and security controls.
IP governance	VPC IPAM	Allocate and audit address space across Regions and accounts.
Transit	AWS Transit Gateway	Connect VPCs, VPNs and Direct Connect through governed route domains.

Capability	Service or pattern	Enterprise use
Private service access	AWS PrivateLink	Expose services without public IPs or broad routing.
Hybrid link	Direct Connect / Site-to-Site VPN	Provide resilient private connectivity to on-premises.
DNS	Route 53 / Resolver	Operate public, private and hybrid name resolution.

Architecture and Operating Decisions

- Use AWS managed services only after recording availability, security, portability and cost consequences.
- Separate production, nonproduction, security and shared services with strong administrative and billing boundaries.
- Prefer short-lived workload identity, private service access and centralized audit evidence over persistent credentials and public administration.
- Automate direct connect, vpn and hybrid routing with reviewed plans, policy tests, staged rollout and post-deployment verification.
- Define recovery objectives, quotas, ownership and decommissioning before the service becomes business critical.

Reference Configuration or Decision Pattern

```

network_standard:
  cidr_source: central_ipam
  topology: regional_transit_gateway
  inspection: dedicated_network_account
  egress: centralized_with_documented_exceptions
  dns: route53_resolver_inbound_outbound
  hybrid: dual_direct_connect_plus_vpn_backup
validation:
  - reachability_analyzer
  - flow_logs
  - dns_query_logging
  - asymmetric_route_test

```

Interpretation: Treat the pattern as a starting point. Replace placeholders with measured requirements, provider limits and approved organizational standards.

Production Readiness Checklist

- ☐ Architecture diagrams and responsibility boundaries for aws networking, dns and hybrid connectivity are current and version controlled.
- ☐ Identity uses groups, roles, workload federation and privileged elevation with no unnecessary standing administrator access.
- ☐ Network paths, DNS behavior, encryption, audit logging and service limits are tested before production use.
- ☐ Changes are delivered through IaC with validation, policy, approval, rollback and drift detection.
- ☐ Recovery, cost allocation, ownership metadata and teardown have measurable evidence.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Deployment succeeds but service is unreachable	Route, firewall, DNS or private-endpoint path is incomplete	Trace the path hop by hop; compare route, policy, DNS and endpoint state; then run an explicit reachability test.	Successful synthetic request and flow/audit evidence
Authorization is broader than intended	Inherited role, wildcard permission or unmanaged credential	Inspect effective permissions, credential age and access logs; replace broad grants with scoped roles and short-lived identity.	Effective-access review and denied-action test
Regional failure breaks the workload	State, dependency or failover mechanism remained single-region	Map stateful dependencies, restore order and traffic controls; exercise failover and fallback against measured RTO/RPO.	Timed recovery log with data-integrity checks

Symptom	Likely cause	Corrective action	Validation evidence
Cost rises without traffic growth	Idle capacity, data transfer, logging volume or commitment mismatch	Break down spend by owner, service, region and usage driver; validate rightsizing or commitment changes against SLOs.	Cost delta plus performance and reliability validation

Hands-on Lab 13: Build an AWS Hub-and-Spoke Network with Private Service Access

Prerequisites

- Disposable provider sandbox or a local simulation.
- Terraform or provider CLI, Git and a diagramming tool.
- Synthetic identities and data; no production credentials or confidential information.

Implementation

1. Create the architecture and responsibility model for aws networking, dns and hybrid connectivity using fictional enterprise requirements.
2. Implement the smallest safe sandbox using Amazon VPC, VPC IPAM, AWS Transit Gateway; use free or low-cost options where available.
3. Apply identity, network, encryption, logging, metadata and budget controls through IaC or an equivalent version-controlled simulation.
4. Run one approved behavior test and one denied behavior test; save the commands, timestamps and resulting telemetry.
5. Inject a bounded configuration or dependency failure, follow the runbook, recover and record measured recovery time.
6. Destroy or quarantine the lab, revoke temporary access and verify that no unexpected billable resources remain.

Validation Evidence

- Version-controlled architecture, IaC and decision record.
- Allowed, denied and failure-test outputs with timestamps.
- Cost, recovery, ownership and teardown verification.

Cleanup

- Destroy test resources in dependency order and verify the provider inventory and billing view.
- Revoke temporary identities, remove local state copies and sanitize all portfolio evidence.

Interview Questions and Model Answers

1. What is the most important design principle in AWS Networking, DNS and Hybrid Connectivity?

Make responsibility and failure boundaries explicit, then automate controls and prove them with normal, denied and recovery tests rather than trusting a successful deployment alone.

2. How would you choose between the main AWS services in this domain?

Start with workload requirements for latency, state, scale, compliance, team capability, recovery and cost. Select the managed service whose operating model fits those requirements and document the consequences in an ADR.

3. What evidence would you require before production approval?

Architecture and threat models, reviewed IaC plan, effective-access results, network and encryption tests, known audit events, SLO and recovery validation, ownership metadata, budget routing and rollback evidence.

4. How do you reduce provider lock-in without creating a lowest-common-denominator platform?

Preserve portable source, data, interfaces and deployment artifacts where a credible exit scenario requires them, while still using provider-specific capabilities when their measurable value exceeds migration and dependency risk.

Chapter Summary

- VPC segmentation and IP address management defines a key architecture boundary.
- Transit Gateway and centralized inspection must be automated and least-privileged.
- Direct Connect, VPN and hybrid routing requires policy, telemetry and recovery evidence.
- Route 53, PrivateLink and controlled ingress/egress should be tested before production and after material change.

Chapter 14: AWS Compute, Containers and Serverless Architecture

Selecting EC2, ECS, EKS, Lambda and managed application platforms

Why this chapter matters: Compute selection determines operational load, scaling behavior, deployment speed and failure modes. The correct service depends on workload constraints, not on a universal modernization slogan.

Learning Objectives

- Explain the architecture, responsibility boundaries and major trade-offs of ec2 and auto scaling operating model.
- Design ecs and eks container platforms for enterprise scale, least privilege and bounded failure.
- Implement lambda event-driven and serverless limits through version-controlled automation and policy checks.
- Troubleshoot deployment, autoscaling and workload isolation using provider-native telemetry and repeatable evidence.
- Deliver a sanitized portfolio artifact for deploy one workload across ec2, ecs or eks and compare trade-offs.

Core Concepts

EC2 and Auto Scaling operating model

EC2 and Auto Scaling operating model is a design boundary, not merely a console feature. In AWS Compute, Containers and Serverless Architecture, the team must document ownership, trust assumptions, dependency direction, failure impact and the evidence used to prove the control. The recommended implementation uses EC2 / Auto Scaling, Amazon ECS, Amazon EKS, AWS Lambda only where they satisfy an explicit requirement; equivalent services are not automatically interchangeable.

ECS and EKS container platforms

A production implementation of ECS and EKS container platforms separates the management plane from workload execution and constrains blast radius. Architecture reviews should evaluate quotas, regional behavior, identity propagation, private connectivity, encryption, logging and recovery. The operational contract must state who changes the control, who monitors it and how an emergency exception expires.

Lambda event-driven and serverless limits

For Lambda event-driven and serverless limits, repeatability matters more than a one-time successful deployment. Infrastructure as code, policy tests, deployment verification, drift detection and rollback evidence should be retained together. A design is incomplete when it works only for the original engineer or depends on undocumented console steps.

Deployment, autoscaling and workload isolation

The strongest validation for Deployment, autoscaling and workload isolation combines an allowed test, a denied test and a failure test. Teams should prove normal workload behavior, demonstrate that prohibited behavior is blocked and exercise recovery under a bounded fault. Metrics, logs, audit events and cost records become part of the acceptance package.

Service and Capability Map

Capability	Service or pattern	Enterprise use
Virtual machines	EC2 / Auto Scaling	Run customizable or legacy workloads with controlled images.
Managed containers	Amazon ECS	Operate containers with lower control-plane overhead.
Kubernetes	Amazon EKS	Use Kubernetes APIs, ecosystem and workload portability.

Capability	Service or pattern	Enterprise use
Functions	AWS Lambda	Run event-driven, short-duration stateless code.
Application platform	App Runner / Elastic Beanstalk	Deploy web services with managed platform operations.
Images	EC2 Image Builder / ECR	Build signed, scanned machine and container artifacts.

Architecture and Operating Decisions

- Use AWS managed services only after recording availability, security, portability and cost consequences.
- Separate production, nonproduction, security and shared services with strong administrative and billing boundaries.
- Prefer short-lived workload identity, private service access and centralized audit evidence over persistent credentials and public administration.
- Automate lambda event-driven and serverless limits with reviewed plans, policy tests, staged rollout and post-deployment verification.
- Define recovery objectives, quotas, ownership and decommissioning before the service becomes business critical.

Reference Configuration or Decision Pattern

```
placement:
  legacy_stateful: ec2_auto_scaling
  simple_containers: ecs_fargate
  kubernetes_ecosystem: eks
  event_handlers: lambda
controls:
  - immutable_images
  - multi_az_capacity
  - workload_identity
  - autoscaling_with_load_test
  - deployment_rollback_alarm
```

Interpretation: Treat the pattern as a starting point. Replace placeholders with measured requirements, provider limits and approved organizational standards.

Production Readiness Checklist

- ☐ Architecture diagrams and responsibility boundaries for aws compute, containers and serverless architecture are current and version controlled.
- ☐ Identity uses groups, roles, workload federation and privileged elevation with no unnecessary standing administrator access.
- ☐ Network paths, DNS behavior, encryption, audit logging and service limits are tested before production use.
- ☐ Changes are delivered through IaC with validation, policy, approval, rollback and drift detection.
- ☐ Recovery, cost allocation, ownership metadata and teardown have measurable evidence.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Deployment succeeds but service is unreachable	Route, firewall, DNS or private-endpoint path is incomplete	Trace the path hop by hop; compare route, policy, DNS and endpoint state; then run an explicit reachability test.	Successful synthetic request and flow/audit evidence
Authorization is broader than intended	Inherited role, wildcard permission or unmanaged credential	Inspect effective permissions, credential age and access logs; replace broad grants with scoped roles and short-lived identity.	Effective-access review and denied-action test
Regional failure breaks the workload	State, dependency or failover mechanism remained single-region	Map stateful dependencies, restore order and traffic controls; exercise failover and fallback against measured RTO/RPO.	Timed recovery log with data-integrity checks

Symptom	Likely cause	Corrective action	Validation evidence
Cost rises without traffic growth	Idle capacity, data transfer, logging volume or commitment mismatch	Break down spend by owner, service, region and usage driver; validate rightsizing or commitment changes against SLOs.	Cost delta plus performance and reliability validation

Hands-on Lab 14: Deploy One Workload Across EC2, ECS or EKS and Compare Trade-offs

Prerequisites

- Disposable provider sandbox or a local simulation.
- Terraform or provider CLI, Git and a diagramming tool.
- Synthetic identities and data; no production credentials or confidential information.

Implementation

1. Create the architecture and responsibility model for aws compute, containers and serverless architecture using fictional enterprise requirements.
2. Implement the smallest safe sandbox using EC2 / Auto Scaling, Amazon ECS, Amazon EKS; use free or low-cost options where available.
3. Apply identity, network, encryption, logging, metadata and budget controls through IaC or an equivalent version-controlled simulation.
4. Run one approved behavior test and one denied behavior test; save the commands, timestamps and resulting telemetry.
5. Inject a bounded configuration or dependency failure, follow the runbook, recover and record measured recovery time.
6. Destroy or quarantine the lab, revoke temporary access and verify that no unexpected billable resources remain.

Validation Evidence

- Version-controlled architecture, IaC and decision record.
- Allowed, denied and failure-test outputs with timestamps.
- Cost, recovery, ownership and teardown verification.

Cleanup

- Destroy test resources in dependency order and verify the provider inventory and billing view.
- Revoke temporary identities, remove local state copies and sanitize all portfolio evidence.

Interview Questions and Model Answers

1. What is the most important design principle in AWS Compute, Containers and Serverless Architecture?

Make responsibility and failure boundaries explicit, then automate controls and prove them with normal, denied and recovery tests rather than trusting a successful deployment alone.

2. How would you choose between the main AWS services in this domain?

Start with workload requirements for latency, state, scale, compliance, team capability, recovery and cost. Select the managed service whose operating model fits those requirements and document the consequences in an ADR.

3. What evidence would you require before production approval?

Architecture and threat models, reviewed IaC plan, effective-access results, network and encryption tests, known audit events, SLO and recovery validation, ownership metadata, budget routing and rollback evidence.

4. How do you reduce provider lock-in without creating a lowest-common-denominator platform?

Preserve portable source, data, interfaces and deployment artifacts where a credible exit scenario requires them, while still using provider-specific capabilities when their measurable value exceeds migration and dependency risk.

Chapter Summary

- EC2 and Auto Scaling operating model defines a key architecture boundary.
- ECS and EKS container platforms must be automated and least-privileged.
- Lambda event-driven and serverless limits requires policy, telemetry and recovery evidence.
- Deployment, autoscaling and workload isolation should be tested before production and after material change.

Chapter 15: AWS Storage, Databases and Data Protection

Matching durability, consistency, throughput and recovery requirements

Why this chapter matters: Data services often dominate migration risk and recovery time. Service choice must include access pattern, consistency, encryption, backup, restore, regional behavior and exit strategy.

Learning Objectives

- Explain the architecture, responsibility boundaries and major trade-offs of object, block and file storage selection.
- Design relational and nosql database architecture for enterprise scale, least privilege and bounded failure.
- Implement backup, replication and retention controls through version-controlled automation and policy checks.
- Troubleshoot data lifecycle, classification and recovery testing using provider-native telemetry and repeatable evidence.
- Deliver a sanitized portfolio artifact for design and test an aws data protection and restore plan.

Core Concepts

Object, block and file storage selection

Object, block and file storage selection is a design boundary, not merely a console feature. In AWS Storage, Databases and Data Protection, the team must document ownership, trust assumptions, dependency direction, failure impact and the evidence used to prove the control. The recommended implementation uses Amazon S3, Amazon EBS, Amazon EFS / FSx, Amazon RDS / Aurora only where they satisfy an explicit requirement; equivalent services are not automatically interchangeable.

Relational and NoSQL database architecture

A production implementation of Relational and NoSQL database architecture separates the management plane from workload execution and constrains blast radius. Architecture reviews should evaluate quotas, regional behavior, identity propagation, private connectivity, encryption, logging and recovery. The operational contract must state who changes the control, who monitors it and how an emergency exception expires.

Backup, replication and retention controls

For Backup, replication and retention controls, repeatability matters more than a one-time successful deployment. Infrastructure as code, policy tests, deployment verification, drift detection and rollback evidence should be retained together. A design is incomplete when it works only for the original engineer or depends on undocumented console steps.

Data lifecycle, classification and recovery testing

The strongest validation for Data lifecycle, classification and recovery testing combines an allowed test, a denied test and a failure test. Teams should prove normal workload behavior, demonstrate that prohibited behavior is blocked and exercise recovery under a bounded fault. Metrics, logs, audit events and cost records become part of the acceptance package.

Service and Capability Map

Capability	Service or pattern	Enterprise use
Object storage	Amazon S3	Durable object data, archives, logs and data lakes.
Block storage	Amazon EBS	Persistent block volumes for EC2 workloads.
File storage	Amazon EFS / FSx	Managed shared file systems for Linux and specialized workloads.
Relational	Amazon RDS / Aurora	Managed relational databases with Multi-AZ and backups.

Capability	Service or pattern	Enterprise use
NoSQL	Amazon DynamoDB	Serverless key-value and document workloads at scale.
Protection	AWS Backup	Central backup policy, vault controls and restore testing.

Architecture and Operating Decisions

- Use AWS managed services only after recording availability, security, portability and cost consequences.
- Separate production, nonproduction, security and shared services with strong administrative and billing boundaries.
- Prefer short-lived workload identity, private service access and centralized audit evidence over persistent credentials and public administration.
- Automate backup, replication and retention controls with reviewed plans, policy tests, staged rollout and post-deployment verification.
- Define recovery objectives, quotas, ownership and decommissioning before the service becomes business critical.

Reference Configuration or Decision Pattern

```
data_class: confidential
storage:
  objects: s3_versioning_object_lock
  block: ebs_encrypted_snapshots
  files: efs_backup_policy
database:
  primary: aurora_multi_az
  replicas: cross_region_if_rto_requires
recovery:
  rpo_minutes: 15
  rto_minutes: 60
  restore_test: quarterly
```

Interpretation: Treat the pattern as a starting point. Replace placeholders with measured requirements, provider limits and approved organizational standards.

Production Readiness Checklist

- ☐ Architecture diagrams and responsibility boundaries for aws storage, databases and data protection are current and version controlled.
- ☐ Identity uses groups, roles, workload federation and privileged elevation with no unnecessary standing administrator access.
- ☐ Network paths, DNS behavior, encryption, audit logging and service limits are tested before production use.
- ☐ Changes are delivered through IaC with validation, policy, approval, rollback and drift detection.
- ☐ Recovery, cost allocation, ownership metadata and teardown have measurable evidence.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Deployment succeeds but service is unreachable	Route, firewall, DNS or private-endpoint path is incomplete	Trace the path hop by hop; compare route, policy, DNS and endpoint state; then run an explicit reachability test.	Successful synthetic request and flow/audit evidence
Authorization is broader than intended	Inherited role, wildcard permission or unmanaged credential	Inspect effective permissions, credential age and access logs; replace broad grants with scoped roles and short-lived identity.	Effective-access review and denied-action test
Regional failure breaks the workload	State, dependency or failover mechanism remained single-region	Map stateful dependencies, restore order and traffic controls; exercise failover and fallback against measured RTO/RPO.	Timed recovery log with data-integrity checks

Symptom	Likely cause	Corrective action	Validation evidence
Cost rises without traffic growth	Idle capacity, data transfer, logging volume or commitment mismatch	Break down spend by owner, service, region and usage driver; validate rightsizing or commitment changes against SLOs.	Cost delta plus performance and reliability validation

Hands-on Lab 15: Design and Test an AWS Data Protection and Restore Plan

Prerequisites

- Disposable provider sandbox or a local simulation.
- Terraform or provider CLI, Git and a diagramming tool.
- Synthetic identities and data; no production credentials or confidential information.

Implementation

1. Create the architecture and responsibility model for aws storage, databases and data protection using fictional enterprise requirements.
2. Implement the smallest safe sandbox using Amazon S3, Amazon EBS, Amazon EFS / FSx; use free or low-cost options where available.
3. Apply identity, network, encryption, logging, metadata and budget controls through IaC or an equivalent version-controlled simulation.
4. Run one approved behavior test and one denied behavior test; save the commands, timestamps and resulting telemetry.
5. Inject a bounded configuration or dependency failure, follow the runbook, recover and record measured recovery time.
6. Destroy or quarantine the lab, revoke temporary access and verify that no unexpected billable resources remain.

Validation Evidence

- Version-controlled architecture, IaC and decision record.
- Allowed, denied and failure-test outputs with timestamps.
- Cost, recovery, ownership and teardown verification.

Cleanup

- Destroy test resources in dependency order and verify the provider inventory and billing view.
- Revoke temporary identities, remove local state copies and sanitize all portfolio evidence.

Interview Questions and Model Answers

1. What is the most important design principle in AWS Storage, Databases and Data Protection?

Make responsibility and failure boundaries explicit, then automate controls and prove them with normal, denied and recovery tests rather than trusting a successful deployment alone.

2. How would you choose between the main AWS services in this domain?

Start with workload requirements for latency, state, scale, compliance, team capability, recovery and cost. Select the managed service whose operating model fits those requirements and document the consequences in an ADR.

3. What evidence would you require before production approval?

Architecture and threat models, reviewed IaC plan, effective-access results, network and encryption tests, known audit events, SLO and recovery validation, ownership metadata, budget routing and rollback evidence.

4. How do you reduce provider lock-in without creating a lowest-common-denominator platform?

Preserve portable source, data, interfaces and deployment artifacts where a credible exit scenario requires them, while still using provider-specific capabilities when their measurable value exceeds migration and dependency risk.

Chapter Summary

- Object, block and file storage selection defines a key architecture boundary.
- Relational and NoSQL database architecture must be automated and least-privileged.
- Backup, replication and retention controls requires policy, telemetry and recovery evidence.
- Data lifecycle, classification and recovery testing should be tested before production and after material change.

Chapter 16: AWS Security Engineering and Detection

Combining preventive controls, threat detection and centralized response

Why this chapter matters: Security services create value only when findings are prioritized, owned and connected to containment. An enterprise design must integrate identity, network, data, configuration and threat telemetry.

Learning Objectives

- Explain the architecture, responsibility boundaries and major trade-offs of encryption, keys and secret lifecycle.
- Design cloudtrail, config and immutable evidence for enterprise scale, least privilege and bounded failure.
- Implement guardduty, security hub and detective controls through version-controlled automation and policy checks.
- Troubleshoot waf, shield and network protection using provider-native telemetry and repeatable evidence.
- Deliver a sanitized portfolio artifact for build an aws security baseline and finding response workflow.

Core Concepts

Encryption, keys and secret lifecycle

Encryption, keys and secret lifecycle is a design boundary, not merely a console feature. In AWS Security Engineering and Detection, the team must document ownership, trust assumptions, dependency direction, failure impact and the evidence used to prove the control. The recommended implementation uses AWS KMS, AWS Secrets Manager, AWS CloudTrail, AWS Config only where they satisfy an explicit requirement; equivalent services are not automatically interchangeable.

CloudTrail, Config and immutable evidence

A production implementation of CloudTrail, Config and immutable evidence separates the management plane from workload execution and constrains blast radius. Architecture reviews should evaluate quotas, regional behavior, identity propagation, private connectivity, encryption, logging and recovery. The operational contract must state who changes the control, who monitors it and how an emergency exception expires.

GuardDuty, Security Hub and detective controls

For GuardDuty, Security Hub and detective controls, repeatability matters more than a one-time successful deployment. Infrastructure as code, policy tests, deployment verification, drift detection and rollback evidence should be retained together. A design is incomplete when it works only for the original engineer or depends on undocumented console steps.

WAF, Shield and network protection

The strongest validation for WAF, Shield and network protection combines an allowed test, a denied test and a failure test. Teams should prove normal workload behavior, demonstrate that prohibited behavior is blocked and exercise recovery under a bounded fault. Metrics, logs, audit events and cost records become part of the acceptance package.

Service and Capability Map

Capability	Service or pattern	Enterprise use
Key management	AWS KMS	Control encryption keys, grants, rotation and audit.
Secrets	AWS Secrets Manager	Rotate and retrieve application secrets through IAM.
Audit	AWS CloudTrail	Record management and selected data events across accounts.
Configuration	AWS Config	Evaluate resource state and organization-wide conformance.

Capability	Service or pattern	Enterprise use
Threat detection	GuardDuty / Security Hub	Aggregate prioritized findings and automate response.
Edge protection	AWS WAF / Shield	Reduce web exploitation and DDoS risk.

Architecture and Operating Decisions

- Use AWS managed services only after recording availability, security, portability and cost consequences.
- Separate production, nonproduction, security and shared services with strong administrative and billing boundaries.
- Prefer short-lived workload identity, private service access and centralized audit evidence over persistent credentials and public administration.
- Automate guardduty, security hub and detective controls with reviewed plans, policy tests, staged rollout and post-deployment verification.
- Define recovery objectives, quotas, ownership and decommissioning before the service becomes business critical.

Reference Configuration or Decision Pattern

```
security_baseline:
  cloudtrail: organization_trail_multi_region
  log_destination: immutable_log_archive
  config: all_supported_resources
  guardduty: delegated_admin_all_accounts
  securityhub: central_configuration
  kms: separation_of_key_admin_and_use
response:
  high_severity: isolate_role_or_resource_then_preserve_evidence
```

Interpretation: Treat the pattern as a starting point. Replace placeholders with measured requirements, provider limits and approved organizational standards.

Production Readiness Checklist

- ☐ Architecture diagrams and responsibility boundaries for aws security engineering and detection are current and version controlled.
- ☐ Identity uses groups, roles, workload federation and privileged elevation with no unnecessary standing administrator access.
- ☐ Network paths, DNS behavior, encryption, audit logging and service limits are tested before production use.
- ☐ Changes are delivered through IaC with validation, policy, approval, rollback and drift detection.
- ☐ Recovery, cost allocation, ownership metadata and teardown have measurable evidence.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Deployment succeeds but service is unreachable	Route, firewall, DNS or private-endpoint path is incomplete	Trace the path hop by hop; compare route, policy, DNS and endpoint state; then run an explicit reachability test.	Successful synthetic request and flow/audit evidence
Authorization is broader than intended	Inherited role, wildcard permission or unmanaged credential	Inspect effective permissions, credential age and access logs; replace broad grants with scoped roles and short-lived identity.	Effective-access review and denied-action test
Regional failure breaks the workload	State, dependency or failover mechanism remained single-region	Map stateful dependencies, restore order and traffic controls; exercise failover and fallback against measured RTO/RPO.	Timed recovery log with data-integrity checks
Cost rises without traffic growth	Idle capacity, data transfer, logging volume or commitment mismatch	Break down spend by owner, service, region and usage driver; validate rightsizing or commitment changes against SLOs.	Cost delta plus performance and reliability validation

Hands-on Lab 16: Build an AWS Security Baseline and Finding Response Workflow

Prerequisites

- Disposable provider sandbox or a local simulation.
- Terraform or provider CLI, Git and a diagramming tool.
- Synthetic identities and data; no production credentials or confidential information.

Implementation

1. Create the architecture and responsibility model for aws security engineering and detection using fictional enterprise requirements.
2. Implement the smallest safe sandbox using AWS KMS, AWS Secrets Manager, AWS CloudTrail; use free or low-cost options where available.
3. Apply identity, network, encryption, logging, metadata and budget controls through IaC or an equivalent version-controlled simulation.
4. Run one approved behavior test and one denied behavior test; save the commands, timestamps and resulting telemetry.
5. Inject a bounded configuration or dependency failure, follow the runbook, recover and record measured recovery time.
6. Destroy or quarantine the lab, revoke temporary access and verify that no unexpected billable resources remain.

Validation Evidence

- Version-controlled architecture, IaC and decision record.
- Allowed, denied and failure-test outputs with timestamps.
- Cost, recovery, ownership and teardown verification.

Cleanup

- Destroy test resources in dependency order and verify the provider inventory and billing view.
- Revoke temporary identities, remove local state copies and sanitize all portfolio evidence.

Interview Questions and Model Answers

1. What is the most important design principle in AWS Security Engineering and Detection?

Make responsibility and failure boundaries explicit, then automate controls and prove them with normal, denied and recovery tests rather than trusting a successful deployment alone.

2. How would you choose between the main AWS services in this domain?

Start with workload requirements for latency, state, scale, compliance, team capability, recovery and cost. Select the managed service whose operating model fits those requirements and document the consequences in an ADR.

3. What evidence would you require before production approval?

Architecture and threat models, reviewed IaC plan, effective-access results, network and encryption tests, known audit events, SLO and recovery validation, ownership metadata, budget routing and rollback evidence.

4. How do you reduce provider lock-in without creating a lowest-common-denominator platform?

Preserve portable source, data, interfaces and deployment artifacts where a credible exit scenario requires them, while still using provider-specific capabilities when their measurable value exceeds migration and dependency risk.

Chapter Summary

- Encryption, keys and secret lifecycle defines a key architecture boundary.
- CloudTrail, Config and immutable evidence must be automated and least-privileged.
- GuardDuty, Security Hub and detective controls requires policy, telemetry and recovery evidence.
- WAF, Shield and network protection should be tested before production and after material change.

Chapter 17: AWS Reliability, Multi-Region Design and Disaster Recovery

Engineering for availability zones, regional impairment and tested recovery

Why this chapter matters: Multi-AZ is not the same as multi-Region, and replication is not the same as recoverability. Reliability architecture must match business RTO/RPO and dependency behavior.

Learning Objectives

- Explain the architecture, responsibility boundaries and major trade-offs of availability zone failure isolation.
- Design multi-region application and data patterns for enterprise scale, least privilege and bounded failure.
- Implement route 53, global accelerator and traffic control through version-controlled automation and policy checks.
- Troubleshoot backup, elastic disaster recovery and recovery exercises using provider-native telemetry and repeatable evidence.
- Deliver a sanitized portfolio artifact for run an aws regional recovery and failback exercise.

Core Concepts

Availability Zone failure isolation

Availability Zone failure isolation is a design boundary, not merely a console feature. In AWS Reliability, Multi-Region Design and Disaster Recovery, the team must document ownership, trust assumptions, dependency direction, failure impact and the evidence used to prove the control. The recommended implementation uses Multi-AZ services / Auto Scaling, Route 53 / Global Accelerator, AWS Application Recovery Controller, Aurora Global Database / S3 replication only where they satisfy an explicit requirement; equivalent services are not automatically interchangeable.

Multi-Region application and data patterns

A production implementation of Multi-Region application and data patterns separates the management plane from workload execution and constrains blast radius. Architecture reviews should evaluate quotas, regional behavior, identity propagation, private connectivity, encryption, logging and recovery. The operational contract must state who changes the control, who monitors it and how an emergency exception expires.

Route 53, Global Accelerator and traffic control

For Route 53, Global Accelerator and traffic control, repeatability matters more than a one-time successful deployment. Infrastructure as code, policy tests, deployment verification, drift detection and rollback evidence should be retained together. A design is incomplete when it works only for the original engineer or depends on undocumented console steps.

Backup, Elastic Disaster Recovery and recovery exercises

The strongest validation for Backup, Elastic Disaster Recovery and recovery exercises combines an allowed test, a denied test and a failure test. Teams should prove normal workload behavior, demonstrate that prohibited behavior is blocked and exercise recovery under a bounded fault. Metrics, logs, audit events and cost records become part of the acceptance package.

Service and Capability Map

Capability	Service or pattern	Enterprise use
Zonal resilience	Multi-AZ services / Auto Scaling	Distribute capacity and state across independent zones.
Traffic	Route 53 / Global Accelerator	Steer users using health, latency and recovery policy.

Capability	Service or pattern	Enterprise use
Recovery control	AWS Application Recovery Controller	Protect routing changes and verify recovery readiness.
Replication	Aurora Global Database / S3 replication	Replicate selected data across Regions.
Server recovery	AWS Elastic Disaster Recovery	Recover supported servers into AWS.
Backup	AWS Backup vaults	Retain isolated recovery points and test restoration.

Architecture and Operating Decisions

- Use AWS managed services only after recording availability, security, portability and cost consequences.
- Separate production, nonproduction, security and shared services with strong administrative and billing boundaries.
- Prefer short-lived workload identity, private service access and centralized audit evidence over persistent credentials and public administration.
- Automate route 53, global accelerator and traffic control with reviewed plans, policy tests, staged rollout and post-deployment verification.
- Define recovery objectives, quotas, ownership and decommissioning before the service becomes business critical.

Reference Configuration or Decision Pattern

```
recovery_tiers:
  tier_0:
    pattern: multi_region_active_active
    rto: minutes
    rpo: near_zero
  tier_1:
    pattern: warm_standby
    rto: <60m
    rpo: <15m
  tier_2:
    pattern: backup_restore
    rto: hours
controls: [route53_health, recovery_readiness, restore_test, failback_plan]
```

Interpretation: Treat the pattern as a starting point. Replace placeholders with measured requirements, provider limits and approved organizational standards.

Production Readiness Checklist

- ☐ Architecture diagrams and responsibility boundaries for aws reliability, multi-region design and disaster recovery are current and version controlled.
- ☐ Identity uses groups, roles, workload federation and privileged elevation with no unnecessary standing administrator access.
- ☐ Network paths, DNS behavior, encryption, audit logging and service limits are tested before production use.
- ☐ Changes are delivered through IaC with validation, policy, approval, rollback and drift detection.
- ☐ Recovery, cost allocation, ownership metadata and teardown have measurable evidence.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Deployment succeeds but service is unreachable	Route, firewall, DNS or private-endpoint path is incomplete	Trace the path hop by hop; compare route, policy, DNS and endpoint state; then run an explicit reachability test.	Successful synthetic request and flow/audit evidence
Authorization is broader than intended	Inherited role, wildcard permission or unmanaged credential	Inspect effective permissions, credential age and access logs; replace broad grants with scoped roles and short-lived identity.	Effective-access review and denied-action test

Symptom	Likely cause	Corrective action	Validation evidence
Regional failure breaks the workload	State, dependency or failover mechanism remained single-region	Map stateful dependencies, restore order and traffic controls; exercise failover and fallback against measured RTO/RPO.	Timed recovery log with data-integrity checks
Cost rises without traffic growth	Idle capacity, data transfer, logging volume or commitment mismatch	Break down spend by owner, service, region and usage driver; validate rightsizing or commitment changes against SLOs.	Cost delta plus performance and reliability validation

Hands-on Lab 17: Run an AWS Regional Recovery and Failback Exercise

Prerequisites

- Disposable provider sandbox or a local simulation.
- Terraform or provider CLI, Git and a diagramming tool.
- Synthetic identities and data; no production credentials or confidential information.

Implementation

1. Create the architecture and responsibility model for aws reliability, multi-region design and disaster recovery using fictional enterprise requirements.
2. Implement the smallest safe sandbox using Multi-AZ services / Auto Scaling, Route 53 / Global Accelerator, AWS Application Recovery Controller; use free or low-cost options where available.
3. Apply identity, network, encryption, logging, metadata and budget controls through IaC or an equivalent version-controlled simulation.
4. Run one approved behavior test and one denied behavior test; save the commands, timestamps and resulting telemetry.
5. Inject a bounded configuration or dependency failure, follow the runbook, recover and record measured recovery time.
6. Destroy or quarantine the lab, revoke temporary access and verify that no unexpected billable resources remain.

Validation Evidence

- Version-controlled architecture, IaC and decision record.
- Allowed, denied and failure-test outputs with timestamps.
- Cost, recovery, ownership and teardown verification.

Cleanup

- Destroy test resources in dependency order and verify the provider inventory and billing view.
- Revoke temporary identities, remove local state copies and sanitize all portfolio evidence.

Interview Questions and Model Answers

1. What is the most important design principle in AWS Reliability, Multi-Region Design and Disaster Recovery?

Make responsibility and failure boundaries explicit, then automate controls and prove them with normal, denied and recovery tests rather than trusting a successful deployment alone.

2. How would you choose between the main AWS services in this domain?

Start with workload requirements for latency, state, scale, compliance, team capability, recovery and cost. Select the managed service whose operating model fits those requirements and document the consequences in an ADR.

3. What evidence would you require before production approval?

Architecture and threat models, reviewed IaC plan, effective-access results, network and encryption tests, known audit events, SLO and recovery validation, ownership metadata, budget routing and rollback evidence.

4. How do you reduce provider lock-in without creating a lowest-common-denominator platform?

Preserve portable source, data, interfaces and deployment artifacts where a credible exit scenario requires them, while still using provider-specific capabilities when their measurable value exceeds migration and dependency risk.

Chapter Summary

- Availability Zone failure isolation defines a key architecture boundary.
- Multi-Region application and data patterns must be automated and least-privileged.
- Route 53, Global Accelerator and traffic control requires policy, telemetry and recovery evidence.
- Backup, Elastic Disaster Recovery and recovery exercises should be tested before production and after material change.

Chapter 18: AWS Observability, Systems Management and Incident Operations

Operating workloads through metrics, logs, traces, inventory and safe automation

Why this chapter matters: Cloud operations require more than dashboards. Teams need consistent telemetry, fleet management, change evidence, alarms tied to user impact and incident runbooks that work under pressure.

Learning Objectives

- Explain the architecture, responsibility boundaries and major trade-offs of cloudwatch metrics, logs and alarms.
- Design opentelemetry, x-ray and service correlation for enterprise scale, least privilege and bounded failure.
- Implement systems manager fleet and automation controls through version-controlled automation and policy checks.
- Troubleshoot incident detection, response and operational evidence using provider-native telemetry and repeatable evidence.
- Deliver a sanitized portfolio artifact for create an aws operations dashboard and incident drill.

Core Concepts

CloudWatch metrics, logs and alarms

CloudWatch metrics, logs and alarms is a design boundary, not merely a console feature. In AWS Observability, Systems Management and Incident Operations, the team must document ownership, trust assumptions, dependency direction, failure impact and the evidence used to prove the control. The recommended implementation uses Amazon CloudWatch, AWS X-Ray / OpenTelemetry, AWS Systems Manager, CloudTrail Lake only where they satisfy an explicit requirement; equivalent services are not automatically interchangeable.

OpenTelemetry, X-Ray and service correlation

A production implementation of OpenTelemetry, X-Ray and service correlation separates the management plane from workload execution and constrains blast radius. Architecture reviews should evaluate quotas, regional behavior, identity propagation, private connectivity, encryption, logging and recovery. The operational contract must state who changes the control, who monitors it and how an emergency exception expires.

Systems Manager fleet and automation controls

For Systems Manager fleet and automation controls, repeatability matters more than a one-time successful deployment. Infrastructure as code, policy tests, deployment verification, drift detection and rollback evidence should be retained together. A design is incomplete when it works only for the original engineer or depends on undocumented console steps.

Incident detection, response and operational evidence

The strongest validation for Incident detection, response and operational evidence combines an allowed test, a denied test and a failure test. Teams should prove normal workload behavior, demonstrate that prohibited behavior is blocked and exercise recovery under a bounded fault. Metrics, logs, audit events and cost records become part of the acceptance package.

Service and Capability Map

Capability	Service or pattern	Enterprise use
Metrics and logs	Amazon CloudWatch	Collect service and application telemetry and alarms.
Tracing	AWS X-Ray / OpenTelemetry	Correlate requests across distributed services.

Capability	Service or pattern	Enterprise use
Fleet operations	AWS Systems Manager	Inventory, patch, run commands and automate operations.
Audit	CloudTrail Lake	Investigate API activity and retain queryable evidence.
Incident workflow	Incident Manager / OpsCenter	Coordinate response and operational work items.
Health	AWS Health / service quotas	Detect provider events and capacity constraints.

Architecture and Operating Decisions

- Use AWS managed services only after recording availability, security, portability and cost consequences.
- Separate production, nonproduction, security and shared services with strong administrative and billing boundaries.
- Prefer short-lived workload identity, private service access and centralized audit evidence over persistent credentials and public administration.
- Automate systems manager fleet and automation controls with reviewed plans, policy tests, staged rollout and post-deployment verification.
- Define recovery objectives, quotas, ownership and decommissioning before the service becomes business critical.

Reference Configuration or Decision Pattern

```
operational_standard:
  telemetry: [metrics, structured_logs, traces, audit]
  alarms: [symptom_based, actionable, owned]
  fleet: systems_manager_managed
  changes: maintenance_windows_and_automation
  incident:
    roles: [commander, operations, communications]
    evidence: timeline_and_api_activity
  game_day: quarterly
```

Interpretation: Treat the pattern as a starting point. Replace placeholders with measured requirements, provider limits and approved organizational standards.

Production Readiness Checklist

- ☐ Architecture diagrams and responsibility boundaries for aws observability, systems management and incident operations are current and version controlled.
- ☐ Identity uses groups, roles, workload federation and privileged elevation with no unnecessary standing administrator access.
- ☐ Network paths, DNS behavior, encryption, audit logging and service limits are tested before production use.
- ☐ Changes are delivered through IaC with validation, policy, approval, rollback and drift detection.
- ☐ Recovery, cost allocation, ownership metadata and teardown have measurable evidence.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Deployment succeeds but service is unreachable	Route, firewall, DNS or private-endpoint path is incomplete	Trace the path hop by hop; compare route, policy, DNS and endpoint state; then run an explicit reachability test.	Successful synthetic request and flow/audit evidence
Authorization is broader than intended	Inherited role, wildcard permission or unmanaged credential	Inspect effective permissions, credential age and access logs; replace broad grants with scoped roles and short-lived identity.	Effective-access review and denied-action test
Regional failure breaks the workload	State, dependency or failover mechanism remained single-region	Map stateful dependencies, restore order and traffic controls; exercise failover and fallback against measured RTO/RPO.	Timed recovery log with data-integrity checks

Symptom	Likely cause	Corrective action	Validation evidence
Cost rises without traffic growth	Idle capacity, data transfer, logging volume or commitment mismatch	Break down spend by owner, service, region and usage driver; validate rightsizing or commitment changes against SLOs.	Cost delta plus performance and reliability validation

Hands-on Lab 18: Create an AWS Operations Dashboard and Incident Drill

Prerequisites

- Disposable provider sandbox or a local simulation.
- Terraform or provider CLI, Git and a diagramming tool.
- Synthetic identities and data; no production credentials or confidential information.

Implementation

1. Create the architecture and responsibility model for aws observability, systems management and incident operations using fictional enterprise requirements.
2. Implement the smallest safe sandbox using Amazon CloudWatch, AWS X-Ray / OpenTelemetry, AWS Systems Manager; use free or low-cost options where available.
3. Apply identity, network, encryption, logging, metadata and budget controls through IaC or an equivalent version-controlled simulation.
4. Run one approved behavior test and one denied behavior test; save the commands, timestamps and resulting telemetry.
5. Inject a bounded configuration or dependency failure, follow the runbook, recover and record measured recovery time.
6. Destroy or quarantine the lab, revoke temporary access and verify that no unexpected billable resources remain.

Validation Evidence

- Version-controlled architecture, IaC and decision record.
- Allowed, denied and failure-test outputs with timestamps.
- Cost, recovery, ownership and teardown verification.

Cleanup

- Destroy test resources in dependency order and verify the provider inventory and billing view.
- Revoke temporary identities, remove local state copies and sanitize all portfolio evidence.

Interview Questions and Model Answers

1. What is the most important design principle in AWS Observability, Systems Management and Incident Operations?

Make responsibility and failure boundaries explicit, then automate controls and prove them with normal, denied and recovery tests rather than trusting a successful deployment alone.

2. How would you choose between the main AWS services in this domain?

Start with workload requirements for latency, state, scale, compliance, team capability, recovery and cost. Select the managed service whose operating model fits those requirements and document the consequences in an ADR.

3. What evidence would you require before production approval?

Architecture and threat models, reviewed IaC plan, effective-access results, network and encryption tests, known audit events, SLO and recovery validation, ownership metadata, budget routing and rollback evidence.

4. How do you reduce provider lock-in without creating a lowest-common-denominator platform?

Preserve portable source, data, interfaces and deployment artifacts where a credible exit scenario requires them, while still using provider-specific capabilities when their measurable value exceeds migration and dependency risk.

Chapter Summary

- CloudWatch metrics, logs and alarms defines a key architecture boundary.
- OpenTelemetry, X-Ray and service correlation must be automated and least-privileged.
- Systems Manager fleet and automation controls requires policy, telemetry and recovery evidence.
- Incident detection, response and operational evidence should be tested before production and after material change.

Chapter 19: AWS FinOps, Governance and Cost Engineering

Connecting account ownership, usage drivers and commitments to business value

Why this chapter matters: Cost optimization is unsafe when it ignores performance, recovery and growth. AWS cost engineering must combine allocation, unit economics, anomaly response and commitment governance.

Learning Objectives

- Explain the architecture, responsibility boundaries and major trade-offs of cost allocation, billing exports and ownership.
- Design budgets, anomaly detection and guardrails for enterprise scale, least privilege and bounded failure.
- Implement savings plans, reservations and capacity commitments through version-controlled automation and policy checks.
- Troubleshoot rightsizing, data transfer and unit economics using provider-native telemetry and repeatable evidence.
- Deliver a sanitized portfolio artifact for build an aws cost allocation and optimization review.

Core Concepts

Cost allocation, billing exports and ownership

Cost allocation, billing exports and ownership is a design boundary, not merely a console feature. In AWS FinOps, Governance and Cost Engineering, the team must document ownership, trust assumptions, dependency direction, failure impact and the evidence used to prove the control. The recommended implementation uses Cost Explorer / Data Exports, Cost allocation tags / account hierarchy, AWS Budgets / Cost Anomaly Detection, Savings Plans / Reserved Instances only where they satisfy an explicit requirement; equivalent services are not automatically interchangeable.

Budgets, anomaly detection and guardrails

A production implementation of Budgets, anomaly detection and guardrails separates the management plane from workload execution and constrains blast radius. Architecture reviews should evaluate quotas, regional behavior, identity propagation, private connectivity, encryption, logging and recovery. The operational contract must state who changes the control, who monitors it and how an emergency exception expires.

Savings Plans, reservations and capacity commitments

For Savings Plans, reservations and capacity commitments, repeatability matters more than a one-time successful deployment. Infrastructure as code, policy tests, deployment verification, drift detection and rollback evidence should be retained together. A design is incomplete when it works only for the original engineer or depends on undocumented console steps.

Rightsizing, data transfer and unit economics

The strongest validation for Rightsizing, data transfer and unit economics combines an allowed test, a denied test and a failure test. Teams should prove normal workload behavior, demonstrate that prohibited behavior is blocked and exercise recovery under a bounded fault. Metrics, logs, audit events and cost records become part of the acceptance package.

Service and Capability Map

Capability	Service or pattern	Enterprise use
Analysis	Cost Explorer / Data Exports	Analyze cost and usage by dimensions and custom reports.
Allocation	Cost allocation tags / account hierarchy	Map spend to product, environment and owner.
Control	AWS Budgets / Cost Anomaly Detection	Route threshold and unexpected-spend alerts.

Capability	Service or pattern	Enterprise use
Commitments	Savings Plans / Reserved Instances	Discount stable eligible usage with managed coverage risk.
Optimization	Compute Optimizer	Generate rightsizing and utilization recommendations.
Governance	Billing Conductor	Model chargeback or showback for billing groups.

Architecture and Operating Decisions

- Use AWS managed services only after recording availability, security, portability and cost consequences.
- Separate production, nonproduction, security and shared services with strong administrative and billing boundaries.
- Prefer short-lived workload identity, private service access and centralized audit evidence over persistent credentials and public administration.
- Automate savings plans, reservations and capacity commitments with reviewed plans, policy tests, staged rollout and post-deployment verification.
- Define recovery objectives, quotas, ownership and decommissioning before the service becomes business critical.

Reference Configuration or Decision Pattern

```
finops_cycle:
  inform: [allocation_coverage, daily_cost, unit_cost]
  optimize: [rightsizing, scheduling, storage_tiering, data_transfer]
  operate: [budget_owner, anomaly_runbook, commitment_review]
commitments:
  purchase_rule: conservative_baseline_only
  review: monthly
  protect: never_trade_slo_for_savings_without_approval
```

Interpretation: Treat the pattern as a starting point. Replace placeholders with measured requirements, provider limits and approved organizational standards.

Production Readiness Checklist

- ☐ Architecture diagrams and responsibility boundaries for aws finops, governance and cost engineering are current and version controlled.
- ☐ Identity uses groups, roles, workload federation and privileged elevation with no unnecessary standing administrator access.
- ☐ Network paths, DNS behavior, encryption, audit logging and service limits are tested before production use.
- ☐ Changes are delivered through IaC with validation, policy, approval, rollback and drift detection.
- ☐ Recovery, cost allocation, ownership metadata and teardown have measurable evidence.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Deployment succeeds but service is unreachable	Route, firewall, DNS or private-endpoint path is incomplete	Trace the path hop by hop; compare route, policy, DNS and endpoint state; then run an explicit reachability test.	Successful synthetic request and flow/audit evidence
Authorization is broader than intended	Inherited role, wildcard permission or unmanaged credential	Inspect effective permissions, credential age and access logs; replace broad grants with scoped roles and short-lived identity.	Effective-access review and denied-action test
Regional failure breaks the workload	State, dependency or failover mechanism remained single-region	Map stateful dependencies, restore order and traffic controls; exercise failover and fallback against measured RTO/RPO.	Timed recovery log with data-integrity checks

Symptom	Likely cause	Corrective action	Validation evidence
Cost rises without traffic growth	Idle capacity, data transfer, logging volume or commitment mismatch	Break down spend by owner, service, region and usage driver; validate rightsizing or commitment changes against SLOs.	Cost delta plus performance and reliability validation

Hands-on Lab 19: Build an AWS Cost Allocation and Optimization Review

Prerequisites

- Disposable provider sandbox or a local simulation.
- Terraform or provider CLI, Git and a diagramming tool.
- Synthetic identities and data; no production credentials or confidential information.

Implementation

1. Create the architecture and responsibility model for aws finops, governance and cost engineering using fictional enterprise requirements.
2. Implement the smallest safe sandbox using Cost Explorer / Data Exports, Cost allocation tags / account hierarchy, AWS Budgets / Cost Anomaly Detection; use free or low-cost options where available.
3. Apply identity, network, encryption, logging, metadata and budget controls through IaC or an equivalent version-controlled simulation.
4. Run one approved behavior test and one denied behavior test; save the commands, timestamps and resulting telemetry.
5. Inject a bounded configuration or dependency failure, follow the runbook, recover and record measured recovery time.
6. Destroy or quarantine the lab, revoke temporary access and verify that no unexpected billable resources remain.

Validation Evidence

- Version-controlled architecture, IaC and decision record.
- Allowed, denied and failure-test outputs with timestamps.
- Cost, recovery, ownership and teardown verification.

Cleanup

- Destroy test resources in dependency order and verify the provider inventory and billing view.
- Revoke temporary identities, remove local state copies and sanitize all portfolio evidence.

Interview Questions and Model Answers

1. What is the most important design principle in AWS FinOps, Governance and Cost Engineering?

Make responsibility and failure boundaries explicit, then automate controls and prove them with normal, denied and recovery tests rather than trusting a successful deployment alone.

2. How would you choose between the main AWS services in this domain?

Start with workload requirements for latency, state, scale, compliance, team capability, recovery and cost. Select the managed service whose operating model fits those requirements and document the consequences in an ADR.

3. What evidence would you require before production approval?

Architecture and threat models, reviewed IaC plan, effective-access results, network and encryption tests, known audit events, SLO and recovery validation, ownership metadata, budget routing and rollback evidence.

4. How do you reduce provider lock-in without creating a lowest-common-denominator platform?

Preserve portable source, data, interfaces and deployment artifacts where a credible exit scenario requires them, while still using provider-specific capabilities when their measurable value exceeds migration and dependency risk.

Chapter Summary

- Cost allocation, billing exports and ownership defines a key architecture boundary.
- Budgets, anomaly detection and guardrails must be automated and least-privileged.
- Savings Plans, reservations and capacity commitments requires policy, telemetry and recovery evidence.
- Rightsizing, data transfer and unit economics should be tested before production and after material change.

Chapter 20: AWS Enterprise Landing-Zone Capstone

Integrating organization, identity, network, workloads, security, operations and cost

Why this chapter matters: The AWS foundation is complete only when a workload account can be vended, connected, governed, observed, recovered and closed through a repeatable product workflow.

Learning Objectives

- Explain the architecture, responsibility boundaries and major trade-offs of account vending and baseline automation.
- Design shared network, identity and security services for enterprise scale, least privilege and bounded failure.
- Implement workload deployment and recovery validation through version-controlled automation and policy checks.
- Troubleshoot operations, cost and lifecycle evidence using provider-native telemetry and repeatable evidence.
- Deliver a sanitized portfolio artifact for deliver the aws enterprise foundation capstone.

Core Concepts

Account vending and baseline automation

Account vending and baseline automation is a design boundary, not merely a console feature. In AWS Enterprise Landing-Zone Capstone, the team must document ownership, trust assumptions, dependency direction, failure impact and the evidence used to prove the control. The recommended implementation uses Control Tower / Organizations, IAM Identity Center, Transit Gateway / Route 53 Resolver, CloudTrail / Config / GuardDuty / Security Hub only where they satisfy an explicit requirement; equivalent services are not automatically interchangeable.

Shared network, identity and security services

A production implementation of Shared network, identity and security services separates the management plane from workload execution and constrains blast radius. Architecture reviews should evaluate quotas, regional behavior, identity propagation, private connectivity, encryption, logging and recovery. The operational contract must state who changes the control, who monitors it and how an emergency exception expires.

Workload deployment and recovery validation

For Workload deployment and recovery validation, repeatability matters more than a one-time successful deployment. Infrastructure as code, policy tests, deployment verification, drift detection and rollback evidence should be retained together. A design is incomplete when it works only for the original engineer or depends on undocumented console steps.

Operations, cost and lifecycle evidence

The strongest validation for Operations, cost and lifecycle evidence combines an allowed test, a denied test and a failure test. Teams should prove normal workload behavior, demonstrate that prohibited behavior is blocked and exercise recovery under a bounded fault. Metrics, logs, audit events and cost records become part of the acceptance package.

Service and Capability Map

Capability	Service or pattern	Enterprise use
Foundation	Control Tower / Organizations	Create the governed multi-account estate.
Identity	IAM Identity Center	Provide federated workforce access.
Network	Transit Gateway / Route 53 Resolver	Connect accounts and hybrid services.
Security	CloudTrail / Config / GuardDuty / Security Hub	Centralize evidence and findings.

Capability	Service or pattern	Enterprise use
Delivery	Terraform and CI OIDC	Provision through reviewed automation.
Cost	Budgets / Data Exports	Allocate and govern spend by owner.

Architecture and Operating Decisions

- Use AWS managed services only after recording availability, security, portability and cost consequences.
- Separate production, nonproduction, security and shared services with strong administrative and billing boundaries.
- Prefer short-lived workload identity, private service access and centralized audit evidence over persistent credentials and public administration.
- Automate workload deployment and recovery validation with reviewed plans, policy tests, staged rollout and post-deployment verification.
- Define recovery objectives, quotas, ownership and decommissioning before the service becomes business critical.

Reference Configuration or Decision Pattern

```
acceptance:
  account_vending_minutes: 30
  long_lived_ci_keys: 0
  public_storage_denied: true
  audit_event_visible_minutes: 5
  regional_recovery_tested: true
  cost_owner_coverage_percent: 100
  teardown_residual_billable_resources: 0
```

Interpretation: Treat the pattern as a starting point. Replace placeholders with measured requirements, provider limits and approved organizational standards.

Production Readiness Checklist

- ☐ Architecture diagrams and responsibility boundaries for aws enterprise landing-zone capstone are current and version controlled.
- ☐ Identity uses groups, roles, workload federation and privileged elevation with no unnecessary standing administrator access.
- ☐ Network paths, DNS behavior, encryption, audit logging and service limits are tested before production use.
- ☐ Changes are delivered through IaC with validation, policy, approval, rollback and drift detection.
- ☐ Recovery, cost allocation, ownership metadata and teardown have measurable evidence.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Deployment succeeds but service is unreachable	Route, firewall, DNS or private-endpoint path is incomplete	Trace the path hop by hop; compare route, policy, DNS and endpoint state; then run an explicit reachability test.	Successful synthetic request and flow/audit evidence
Authorization is broader than intended	Inherited role, wildcard permission or unmanaged credential	Inspect effective permissions, credential age and access logs; replace broad grants with scoped roles and short-lived identity.	Effective-access review and denied-action test
Regional failure breaks the workload	State, dependency or failover mechanism remained single-region	Map stateful dependencies, restore order and traffic controls; exercise failover and fallback against measured RTO/RPO.	Timed recovery log with data-integrity checks
Cost rises without traffic growth	Idle capacity, data transfer, logging volume or commitment mismatch	Break down spend by owner, service, region and usage driver; validate rightsizing or commitment changes against SLOs.	Cost delta plus performance and reliability validation

Hands-on Lab 20: Deliver the AWS Enterprise Foundation Capstone

Prerequisites

- Disposable provider sandbox or a local simulation.
- Terraform or provider CLI, Git and a diagramming tool.
- Synthetic identities and data; no production credentials or confidential information.

Implementation

1. Create the architecture and responsibility model for aws enterprise landing-zone capstone using fictional enterprise requirements.
2. Implement the smallest safe sandbox using Control Tower / Organizations, IAM Identity Center, Transit Gateway / Route 53 Resolver; use free or low-cost options where available.
3. Apply identity, network, encryption, logging, metadata and budget controls through IaC or an equivalent version-controlled simulation.
4. Run one approved behavior test and one denied behavior test; save the commands, timestamps and resulting telemetry.
5. Inject a bounded configuration or dependency failure, follow the runbook, recover and record measured recovery time.
6. Destroy or quarantine the lab, revoke temporary access and verify that no unexpected billable resources remain.

Validation Evidence

- Version-controlled architecture, IaC and decision record.
- Allowed, denied and failure-test outputs with timestamps.
- Cost, recovery, ownership and teardown verification.

Cleanup

- Destroy test resources in dependency order and verify the provider inventory and billing view.
- Revoke temporary identities, remove local state copies and sanitize all portfolio evidence.

Interview Questions and Model Answers

1. What is the most important design principle in AWS Enterprise Landing-Zone Capstone?

Make responsibility and failure boundaries explicit, then automate controls and prove them with normal, denied and recovery tests rather than trusting a successful deployment alone.

2. How would you choose between the main AWS services in this domain?

Start with workload requirements for latency, state, scale, compliance, team capability, recovery and cost. Select the managed service whose operating model fits those requirements and document the consequences in an ADR.

3. What evidence would you require before production approval?

Architecture and threat models, reviewed IaC plan, effective-access results, network and encryption tests, known audit events, SLO and recovery validation, ownership metadata, budget routing and rollback evidence.

4. How do you reduce provider lock-in without creating a lowest-common-denominator platform?

Preserve portable source, data, interfaces and deployment artifacts where a credible exit scenario requires them, while still using provider-specific capabilities when their measurable value exceeds migration and dependency risk.

Chapter Summary

- Account vending and baseline automation defines a key architecture boundary.
- Shared network, identity and security services must be automated and least-privileged.
- Workload deployment and recovery validation requires policy, telemetry and recovery evidence.
- Operations, cost and lifecycle evidence should be tested before production and after material change.

AWS Enterprise Architecture and Operations Reference

Use this reference to review architecture outcomes, implementation controls and the evidence required for production approval.

Domain	Required outcome	Validation evidence
Hierarchy and ownership	Durable resource boundaries with accountable owners	Inventory, effective policy and lifecycle tests
Identity	Federated humans and keyless workloads	Token, role, access-review and denied-action evidence
Network	Approved routes, DNS, private access and egress	Reachability, flow, DNS, latency and failover tests
Data	Classified, encrypted and recoverable state	Backup, restore, reconciliation and exit evidence
Security	Preventive, detective and recovery controls	Policy results, findings, containment and audit timeline
Operations	SLOs, telemetry, runbooks and incident process	Dashboards, alerts, drill timeline and postmortem
FinOps	Allocation, budgets, unit economics and commitment governance	Billing export, owner coverage and optimization record
Delivery	Reviewed IaC and immutable artifact promotion	Plan, policy, signature, deployment and rollback chain
Reliability	Measured capacity, RTO/RPO and failback	Load, failure, recovery and integrity tests
Lifecycle	Vending, upgrade, exception and closure	Fresh deployment, drift and zero-residual teardown

Completion Checklist

- ☐ I can explain the architecture and trade-offs in every chapter of 12B.
- ☐ I can implement at least one sandbox pattern through IaC and short-lived identity.
- ☐ I can demonstrate approved, denied and failure behavior with provider-native evidence.
- ☐ I can design private connectivity, data protection, recovery and cost governance.
- ☐ I can produce architecture diagrams, ADRs, runbooks and troubleshooting timelines.
- ☐ I can destroy the lab safely and prove that no unexpected billable resource remains.

Additional Portfolio Projects

1. Resource vending workflow with ownership, policy, budget and closure automation.
2. Keyless CI/CD identity module and effective-access test harness.
3. Private network and DNS validation pack with route and failure scenarios.
4. Managed data backup, restore and reconciliation project.
5. Security finding triage and automated containment simulation.
6. SLO dashboard and incident-command game day.
7. Billing export, allocation coverage and unit-cost dashboard.
8. IaC provider-upgrade and drift-remediation pipeline.
9. Regional or cross-provider disaster-recovery exercise.
10. Executive architecture brief with decisions, risks, roadmap and measured outcomes.

30-Day Practice Plan

Days	Focus	Required output
1-4	Hierarchy, ownership and identity	Architecture diagram, RACI and access tests
5-8	Networking and private services	IP plan, flow matrix and reachability evidence
9-12	Compute and delivery	Workload deployment, autoscaling and rollback
13-16	Data and recovery	Backup, restore and integrity report
17-20	Security and audit	Threat model, policy tests and finding workflow
21-23	Observability and incidents	SLO, dashboard and incident timeline
24-26	FinOps and governance	Allocation, budget and unit-cost review
27-28	Failure drills	Bounded outage and recovery evidence
29-30	Capstone and portfolio	Architecture defense, cleanup and sanitized report

Glossary

Term	Definition
Application landing zone	A governed environment in which a workload team deploys and operates applications.
Blast radius	The maximum scope of impact from a failure, credential or change.
Control plane	Interfaces and services used to administer a cloud or platform.
Data plane	The runtime path that processes application traffic and data.
Federation	Trust that allows an external identity source to authenticate without duplicate long-lived credentials.
Guardrail	A preventive, detective or corrective control that constrains activity.
Landing zone	A scalable cloud foundation for hierarchy, identity, network, security, logging and cost.
Private endpoint	A private network interface used to access a managed service without a public path.
RPO	Maximum acceptable data loss measured in time.
RTO	Target time to restore a service after disruption.
Unit cost	Technology cost per useful business or technical output.
Workload identity	A short-lived identity issued to software based on trusted runtime context.

Official Documentation Index

The following official sources were checked for this 2026 edition. Verify current service behavior, quotas, prices and certification requirements before production use or exam registration.

- [AWS Control Tower User Guide](#)
- [AWS Organizations User Guide](#)
- [AWS IAM Best Practices](#)
- [AWS VPC Transit Gateway](#)
- [AWS Well-Architected Framework](#)
- [AWS Security Reference Architecture](#)
- [AWS Disaster Recovery of Workloads](#)
- [AWS Cloud Financial Management](#)

Volume 12B completion: preserve your architecture, code, test evidence, recovery timeline, cost review and sanitized final report as one portfolio package.